

AI Interview Prep Simulator

An AI-Driven System for Personalized, Job-Posting-Based Interview Practice

Phase 1 – Conception Phase Report

Course: CSEMCPSE01 – Project: Software Engineering

Student Name: Elijah Kapenga

Matriculation Number: 10254681

Tutor: Dr. Naghmeh Niknejad

Revised version — incorporates tutor feedback from the Phase 1 review

Submission Date: August 13, 2026

Concept Summary

The AI Interview Prep Simulator is a web-based system that helps job seekers prepare for interviews using real job postings. A candidate uploads a resume and either searches for a live job posting through a job search API (Adzuna or The Muse) or enters a job description manually. The system extracts the key role and skill information, generates a tailored set of behavioral and technical interview questions using a large language model (LLM), and lets the candidate answer by typing or by recording a spoken answer, which is transcribed automatically to text. Each answer is evaluated by the LLM against relevance, clarity, and use of concrete examples, and the candidate receives structured, per-question feedback plus an overall readiness score and an exportable PDF report.

The project's objective is to design and implement this end-to-end practice flow as a single-developer, time-boxed software engineering project. Its scope covers one resume and one job posting per session, job search or manual job-description entry, tailored question generation, text and voice answer capture, AI-based scoring and feedback, and a downloadable report; it explicitly excludes video-based interview simulation, multi-user panels, persistent accounts, and any direct LinkedIn integration, which its terms of service prohibit.

The target users are individual job seekers — particularly students, recent graduates, and early-career professionals switching roles or industries — who lack accessible, role-specific interview practice with objective feedback, plus career-services staff who could point clients toward the tool. The customer value is faster, more targeted interview preparation than generic question banks or unstructured self-practice, tied to a real job posting rather than a hypothetical one.

Development follows the Waterfall model, chosen because the project has a small, well-bounded scope centered on one core user flow, a single developer (removing cross-phase coordination overhead), a fixed set of external API integrations whose interfaces are known upfront, and a short, firm submission deadline — conditions that favor a linear, milestone-gated process over an iterative one for this project.

Table of Contents

Concept Summary.....	2
List of Figures and Tables.....	4
1. Project Definition and Project Profile.....	5
2. Risks and Mitigation Strategies.....	7
3. Software Development Methodology.....	9
4. Requirements Specification.....	10
5. Software Development Planning.....	13
6. Requirements Analysis Models.....	14
7. System Design and Architecture.....	16
8. Technologies and Tools.....	18
9. Testing Approach.....	19
10. Glossary.....	20
11. References.....	21

List of Figures and Tables

List of Figures

Figure 1: Software Development Plan – Gantt Chart (Section 5)

Figure 2: Use Case Diagram (UML notation) (Section 6)

Figure 3: Domain Model (UML notation) (Section 6)

Figure 4: System Context Diagram (C4 model, Level 1: System Context) (Section 7)

Figure 5: Component Diagram / Building Block View (C4 model, Level 2/3: Component — informal, adapted for a single-service architecture) (Section 7)

List of Tables

Table 1: Target Users and Customer Value (Section 1)

Table 2: Project Organization (Section 1)

Table 3: Project Technical Risks and Mitigation Strategies (Section 2)

Table 4: Project Business Risks and Mitigation Strategies (Section 2)

Table 5: Functional Requirements (Section 4)

Table 6: Non-Functional Requirements (Section 4)

Table 7: Software Development Plan – Work Packages (Section 5)

Table 8: Technologies and Tools (Section 8)

Table 9: Test Cases (Section 9)

Table 10: Glossary (Section 10)

1. Project Definition and Project Profile

1.1 Project Background and Problem Statement

What is the problem? Job seekers preparing for interviews typically rely on generic question banks or unstructured self-practice, neither of which reflects the specific job posting they are applying for, and neither of which provides objective feedback on their answers.

Who experiences this problem? Individual job seekers, particularly students, recent graduates, and early-career professionals, who do not have regular access to a human mock-interviewer, career coach, or structured interview-preparation tooling.

Why is this problem important? Interview performance is a major determinant of hiring outcomes. According to LinkedIn's 2026 research, 80% of professionals feel unprepared to find a job, spanning every generation, industry, and seniority level (LinkedIn, 2026). Separately, 92% of candidates consider mock-interview practice crucial to preparation (Careerflow, n.d.), and industry data indicates that 89% of new-hire failures stem from attitude or interpersonal issues rather than technical ability (RecruitBPM, 2026) — underscoring that how a candidate communicates and prepares matters as much as what they know. Preparation that is not tailored to a specific role limits a candidate's ability to demonstrate fit for that role.

What happens if the problem is not solved? Candidates continue to enter interviews with generic, unfocused preparation, leading to weaker performance, lower confidence, and reduced chances of receiving offers — particularly relative to candidates with access to paid coaching or structured practice tools.

1.2 Project Objective and Goal

Objective: To design and implement a web-based system that lets a candidate find a real job posting, then simulates an interview tailored to that posting and their resume, using an LLM to generate role-specific questions and score the candidate's answers.

Goal: The expected final result is a working web application in which a candidate can complete an end-to-end interview-practice session — from resume upload and job search through to a scored, exportable feedback report — for a self-selected, real job posting, within the constraints of a single-developer, time-boxed university project.

1.3 Project Scope

Included in scope:

- Single resume vs. single job posting per practice session
- PDF/DOCX resume upload; job postings sourced via a job search API or entered manually
- LLM-generated interview questions (behavioral and technical)
- Text and voice answer capture, with speech-to-text transcription
- LLM-based answer evaluation, per-question feedback, and an overall readiness score
- In-session history and PDF export of the feedback report

Excluded from scope:

- Real-time video interview simulation or facial/body-language analysis
- Multi-user interview panels or live human-in-the-loop interviewing
- Persistent user accounts or long-term storage of resumes/recordings across sessions
- Integration with real recruiting platforms or applicant tracking systems
- Any direct scraping of or integration with LinkedIn, which its terms of service prohibit

Note on AI/LLM Usage: This is a software engineering project, not a machine-learning research project. The system does not train, fine-tune, or evaluate any machine learning model. It integrates third-party AI services — an LLM API and a speech-to-text API — as external dependencies in the same way it integrates the job search API, comparable to how a typical system might integrate a payment gateway or email service. The engineering

focus is the end-to-end pipeline: API integration, data flow between modules, a session/data model, a tested web UI, and traceable requirements, not model development.

1.4 Target Users and Customer Value

Table 1 defines the target user groups for this system, the reason each group was selected, and the value the system provides to them.

Target User Group	Description of User Group	Reason for Targeting This Group	Customer Value Provided
Students & recent graduates	Final-year students or graduates applying for their first full-time roles, often with limited interview experience	This group faces interviews most frequently during job-search season and has the least access to professional coaching	Role-specific practice questions and objective feedback that build interview confidence without paid coaching
Early-career job seekers (career switchers)	Professionals with 1–5 years of experience applying to a new role or industry	This group must reframe existing experience for unfamiliar job descriptions, which generic question banks do not support	Tailored questions and feedback based on the actual job posting they are targeting, helping align answers to the new role
Career services / bootcamp coaches (secondary)	Staff who support job seekers with interview preparation as part of their program	This group needs a scalable tool to supplement one-on-one coaching sessions with structured practice	A ready-to-use practice tool they can point students or clients toward, without running every mock interview personally

Table 1: Target Users and Customer Value

1.5 Project Organization

As this is an individually developed project, all roles below are self-assigned to the author; the responsibilities of each role are nonetheless kept distinct to reflect a realistic software engineering workflow.

Role	Responsible Person	Responsibility
Project Manager	Elijah Kapenga	Plans and tracks the project timeline, manages scope and milestones
Requirements Analyst	Elijah Kapenga	Defines and documents functional and non-functional requirements
System / Software Architect	Elijah Kapenga	Designs system architecture, component structure, and the domain/data model
Backend Developer	Elijah Kapenga	Implements API integrations, business logic, and the session/database layer
Frontend Developer	Elijah Kapenga	Implements the web UI and the answer-capture flow (text and voice)
QA / Tester	Elijah Kapenga	Designs and executes test cases, verifies that requirements are met
Technical Writer	Elijah Kapenga	Produces project documentation, including this report

Table 2: Project Organization

1.6 Comparison with Existing Interview-Preparation Tools

Several categories of existing tools address parts of interview preparation but not the combination this project targets. Peer-to-peer practice platforms (e.g., Pramp) pair candidates for mutual mock interviews but involve no AI-based scoring of answers. Delivery-coaching tools (e.g., Yoodli) analyze speech delivery — pace, filler words, eye contact — but assume the content of the answer is already correct and do not evaluate it against a specific job posting. Live-interview-copilot tools (e.g., Final Round AI) assist candidates during an actual interview rather than preparing them beforehand. Course-style platforms (e.g., Big Interview) provide a library of generic, pre-written questions rather than questions generated from a specific job posting and the candidate's own resume. No reviewed tool combines job-posting-specific question generation, resume-aware tailoring, dual-mode (text/voice) answer capture, and objective per-answer AI scoring in one self-contained session. This is the distinct benefit the AI Interview Prep Simulator provides to its target users: faster, more targeted preparation for a specific application, with feedback that is objective and role-matched rather than generic.

2. Risks and Mitigation Strategies

Risks are grouped into technical risks, which relate to the system's implementation, and business/project risks, which relate to scope, schedule, and external dependencies. Probability and impact are each classified as Low, Medium, or High.

2.1 Technical Risks

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
TR-01	The LLM produces inconsistent or low-quality evaluation output (e.g., vague or hallucinated feedback)	Medium	High	Use a fixed evaluation rubric in the prompt template; spot-check LLM feedback against a small set of manually scored sample answers before the demo
TR-02	Speech-to-text transcription is inaccurate for accented or low-quality audio	Medium	Medium	Display the transcribed text to the user before submission so they can confirm or edit it; allow falling back to text input if transcription fails
TR-03	The job search API (Adzuna/The Muse) has rate limits, downtime, or returns incomplete data	Medium	Medium	Provide a manual job-description entry fallback (FR-05); cache the last successful search results within the session
TR-04	LLM or speech-to-text API usage costs exceed the available budget during development or testing	Low	Medium	Cap the number and length of LLM calls per session (NFR-07); use free-tier/sandbox keys during development and reserve paid calls for final testing and the demo
TR-05	Sensitive candidate data (resumes, job data, or voice recordings) is exposed, retained longer than necessary, or mishandled by a third-party API	Medium	High	Send only the minimum text/audio required over HTTPS to each third-party API; do not persist resumes, audio, or transcripts beyond the active session, consistent with NFR-03; store API keys server-side only, consistent with NFR-06; before final provider selection, review each LLM/speech-to-text provider's data-retention and

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
				model-training policy and prefer providers offering zero-retention or no-training options for API traffic.

Table 3: Project Technical Risks and Mitigation Strategies

2.2 Business / Project Risks

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
BR-01	Schedule overrun given the short, fixed submission deadline	Medium	High	Follow the fixed Waterfall build order in Section 5 with weekly self-checks against the plan; de-scope Could-Have requirements first if behind schedule
BR-02	Scope creep from adding features beyond the agreed Must-Have/Should-Have requirements	Medium	Medium	Enforce the MoSCoW prioritization in Section 4; log new feature ideas but do not implement them until Must-Have items are complete
BR-03	Sole-developer availability risk (illness, other coursework deadlines) reduces available development time	Low	High	Build schedule buffer into each work package (Section 5); prioritize Must-Have requirements early so a minimum viable system exists even under time pressure
BR-04	Reliance on external, third-party APIs (job search, LLM, speech-to-text) whose terms of service or pricing could change during development	Low	Medium	Select providers with stable, well-documented developer APIs; isolate each integration behind its own module (Section 7) so a provider can be swapped with limited impact

Table 4: Project Business Risks and Mitigation Strategies

3. Software Development Methodology

This project follows the Waterfall model, progressing sequentially through Requirements Analysis, System Design, Implementation, Testing and Evaluation, and Deployment/Documentation, as detailed in the work packages in Section 5.

Justification for this project specifically:

- Well-bounded scope: the project centers on a single core user flow (search or enter a job posting → generate questions → answer → receive feedback), so requirements are largely fixed once the architecture is agreed, rather than emerging through ongoing stakeholder discovery.
- Single developer: as the sole team member, there is no cross-functional team to coordinate through sprint ceremonies (stand-ups, sprint planning, retrospectives); the overhead that makes Scrum valuable for teams would add process cost without benefit here.
- Fixed external interfaces: the job search API, LLM API, and speech-to-text API each have stable, documented interfaces that are selected during the design phase and do not change iteratively during development.
- Short, firm deadline: with a fixed submission date and no opportunity for multiple release cycles, a linear model with clear phase gates (each producing a defined, reviewable deliverable) is easier to track against a single deadline than an iterative backlog.

By contrast, Kanban or Scrum are better suited to projects with evolving requirements, multiple contributors, or ongoing stakeholder feedback loops — none of which apply to this single-developer, fixed-scope, time-boxed project. Waterfall's sequential gates (Section 5, Table 7 and Figure 1) also align naturally with the phase-based structure of this course, where each phase is reviewed before the next begins.

4. Requirements Specification

Each requirement has a unique ID, used consistently throughout this document, in the work-package table (Section 5), and in the test cases (Section 9).

4.1 Functional Requirements

Functional requirements describe what the system must do. Priorities use the MoSCoW method (Must Have, Should Have, Could Have, Won't Have This Time).

Req. ID	Requirement Description	Priority	Related Actor/User	Acceptance Criterion
FR-01	The system shall allow a user to upload a resume in PDF or DOCX format.	Must Have	Candidate	A valid PDF/DOCX file is accepted and its text is extracted within 5 seconds.
FR-02	The system shall allow a user to search for job postings by title/keyword using a job search API.	Must Have	Candidate	Given a keyword, the system returns at least one matching posting within 5 seconds.
FR-03	The system shall display a list of matching job postings (title, company, short summary).	Should Have	Candidate	Each result in the list shows a title, company name, and short summary.
FR-04	The system shall allow a user to select a job posting from search results to auto-populate the job description.	Must Have	Candidate	Selecting a result populates the job description field without manual copy-paste.
FR-05	The system shall allow a user to manually paste or upload a job description as an alternative to searching.	Must Have	Candidate	A pasted or uploaded job description is accepted and used in place of a search result.
FR-06	The system shall extract role and skill information from the resume and job description.	Must Have	Candidate	At least 5 key skills/terms are identified from the combined resume and job-description text.
FR-07	The system shall generate a tailored set of interview questions using an LLM API.	Must Have	Candidate	At least 5 questions are returned, mixing behavioral and technical types.
FR-08	The system shall present interview questions to the user one at a time in a structured session flow.	Should Have	Candidate	The UI shows one question at a time with a visible progress indicator (e.g., 3/6).
FR-09	The system shall allow the user to answer each question via typed text input.	Must Have	Candidate	The user can type and submit a text answer for the current question.
FR-10	The system shall allow the user to answer via voice recording, transcribed to text.	Should Have	Candidate	Recorded audio is transcribed to text and shown to the user

Req. ID	Requirement Description	Priority	Related Actor/User	Acceptance Criterion
				before submission.
FR-11	The system shall evaluate each answer using an LLM API (relevance, clarity, use of examples).	Must Have	Candidate	Each submitted answer receives a numeric score and written feedback within 15 seconds.
FR-12	The system shall display per-question feedback, including improvement suggestions.	Must Have	Candidate	Feedback text appears immediately after evaluation, including at least one suggestion.
FR-13	The system shall compute and display an overall interview readiness score for the session.	Must Have	Candidate	An aggregate score (0–100) is shown at the end of the session.
FR-14	The system shall display the session's questions, answers, and scores for review before it ends.	Should Have	Candidate	The user can review all questions, answers, and scores on one screen before exporting.
FR-15	The system shall allow the user to export a feedback report as a PDF.	Should Have	Candidate	The user can download a PDF containing questions, answers, scores, and suggestions.
FR-16	The system shall validate uploaded files/audio and show clear error messages on failure.	Must Have	Candidate	Unsupported file types or failed API calls show a clear, non-technical error message.
FR-17	The system shall allow the user to start a new session without restarting the application.	Could Have	Candidate	The user can begin a new session from within the running application.

Table 5: Functional Requirements

4.2 Non-Functional Requirements

Non-functional requirements describe quality attributes of the system and are measurable or verifiable where possible.

Req. ID	Category	Requirement Description	Measurement / Verification Method
NFR-01	Performance	Question generation completes within 10 seconds; answer feedback within 15 seconds.	Measured via timestamps logged during manual test execution.
NFR-02	Reliability	The system handles LLM/API/transcription failures gracefully: in at least 95% of simulated failure scenarios, a clear user-facing error message is returned within 2 seconds instead of crashing.	Verified via fault-injection test cases that simulate API timeouts and errors.
NFR-03	Privacy	Resumes, job data, and audio recordings are not permanently stored beyond the active	Verified by inspecting server storage/database contents

Req. ID	Category	Requirement Description	Measurement / Verification Method
		session: no candidate data persists more than 30 minutes after a session ends or a server restart.	after a session ends.
NFR-04	Compliance	Job data is sourced only via the job search API's terms of use; no scraping of LinkedIn or other platforms.	Verified via code review of the job-sourcing module.
NFR-05	Usability	The interview flow requires no more than three user actions to reach the first question.	Verified via a manual usability walkthrough.
NFR-06	Security	API keys/credentials are stored server-side only (environment variables), never exposed to the client.	Verified by inspecting client-side network requests and source code.
NFR-07	Cost control	LLM calls are capped at a maximum of 20 calls per browser session (configurable) to bound usage cost.	Verified via request-count logging per session.
NFR-08	Portability	The system runs in any modern browser with microphone access, with no client-side install.	Verified via manual testing on Chrome, Firefox, and Edge.
NFR-09	Maintainability	The codebase is organized into independent modules matching the component diagram (Figure 5), with a 1:1 mapping between source modules and all 9 components shown.	Verified via code-structure review against Figure 5.

Table 6: Non-Functional Requirements

5. Software Development Planning

Table 7 shows the main work packages for moving from conception to a tested, deployed system, with planned start/end dates, the related module, and related requirement IDs. Figure 1 presents the same plan as a Gantt chart, showing sequence and duration; work packages are executed sequentially, so each depends on completion of the previous one.

WP ID	Description	Start Date	End Date	Related Module	Related Requirement IDs
WP-01	Requirements finalization & system design	Jul 23	Jul 29	Design	—
WP-02	Resume/JD ingestion & parser	Jul 30	Aug 2	Ingestion Module	FR-01, FR-05, FR-06
WP-03	Job search API integration	Aug 3	Aug 5	Job Search Module	FR-02, FR-03, FR-04
WP-04	LLM question-generation pipeline	Aug 6	Aug 8	Question Generator	FR-06, FR-07, FR-08
WP-05	Answer capture (text + voice) & session UI	Aug 9	Aug 11	Web UI	FR-09, FR-10
WP-06	Speech-to-text + LLM evaluation & scoring	Aug 12	Aug 15	Transcriber / Evaluator	FR-10, FR-11, FR-12, FR-13
WP-07	Feedback display, history & PDF export	Aug 16	Aug 19	Feedback / Reporting	FR-12, FR-14, FR-15
WP-08	Integration testing & bug fixing	Aug 20	Aug 22	All Modules	FR-16, NFR-01–NFR-09
WP-09	Deployment & final documentation	Aug 23	Aug 25	Deployment	—

Table 7: Software Development Plan – Work Packages

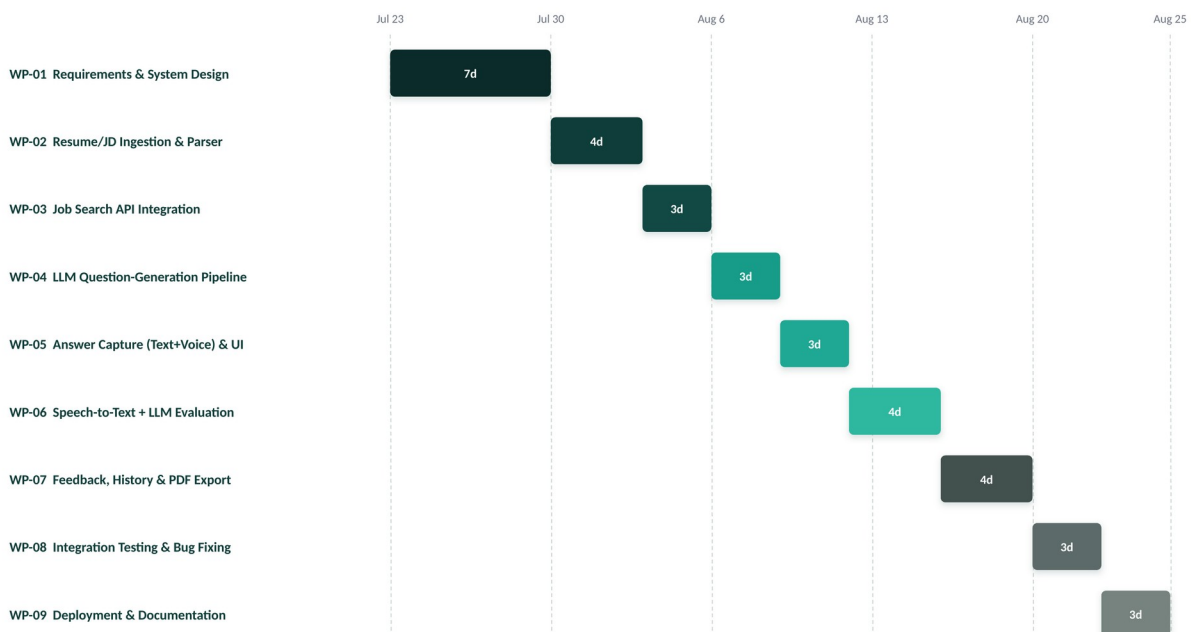


Figure 1: Software Development Plan – Gantt Chart

6. Requirements Analysis Models

6.1 Use Case Diagram

Figure 2 shows the use case diagram for the system. The candidate is the only actor, since this is a single-user, single-session tool with no administrator or second human role in scope. The six use cases correspond directly to the core functional requirements: uploading a resume and selecting or searching for a job posting (FR-01–FR-05), extracting role and skill requirements (FR-06), generating tailored questions (FR-07), answering via text or voice (FR-09, FR-10), receiving AI-scored feedback (FR-11–FR-13), and exporting a PDF readiness report (FR-15).

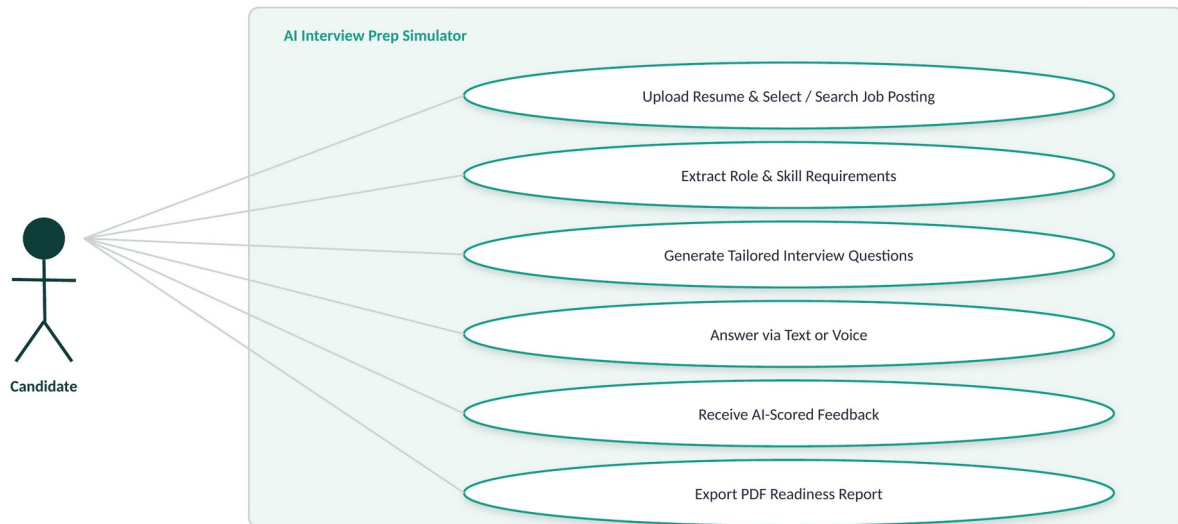


Figure 2: Use Case Diagram (UML notation)

6.2 Domain Model

Figure 3 shows the domain model: the key entities in the system's problem domain and their relationships. A Candidate initiates one or more InterviewSession instances; each session is linked to exactly one Resume and one JobPosting, and contains multiple Question entities. Each Question has exactly one Answer, which in turn has exactly one Feedback entry produced by the LLM evaluator. A session's Feedback entries are aggregated into a single ReadinessReport, which stores the overall score and the exported PDF path.

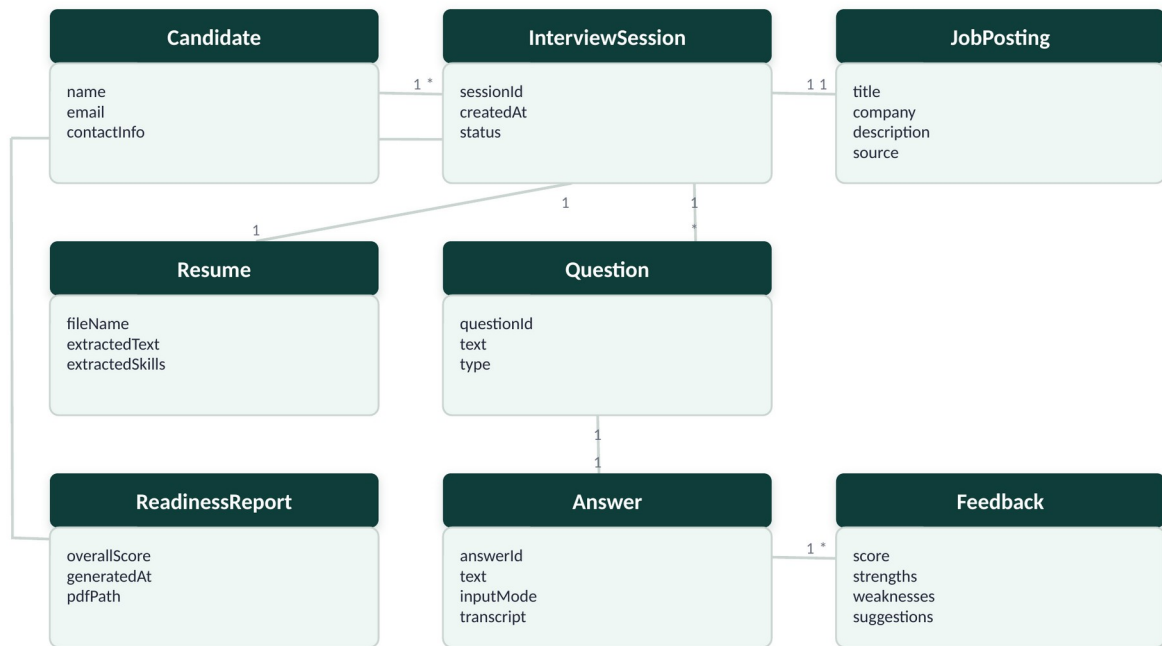


Figure 3: Domain Model (UML notation)

7. System Design and Architecture

7.1 System Context Diagram

Figure 4 shows the system as a single unit and its relationships with external users and systems. The Candidate is the only external user. Three external APIs are involved: the Job Search API (Adzuna or The Muse) supplies real job postings; the LLM API (OpenAI or Claude) generates interview questions and evaluates answers; and the Speech-to-Text API (Whisper) transcribes voice answers to text. The system also produces a PDF feedback report as an output artifact for the candidate.

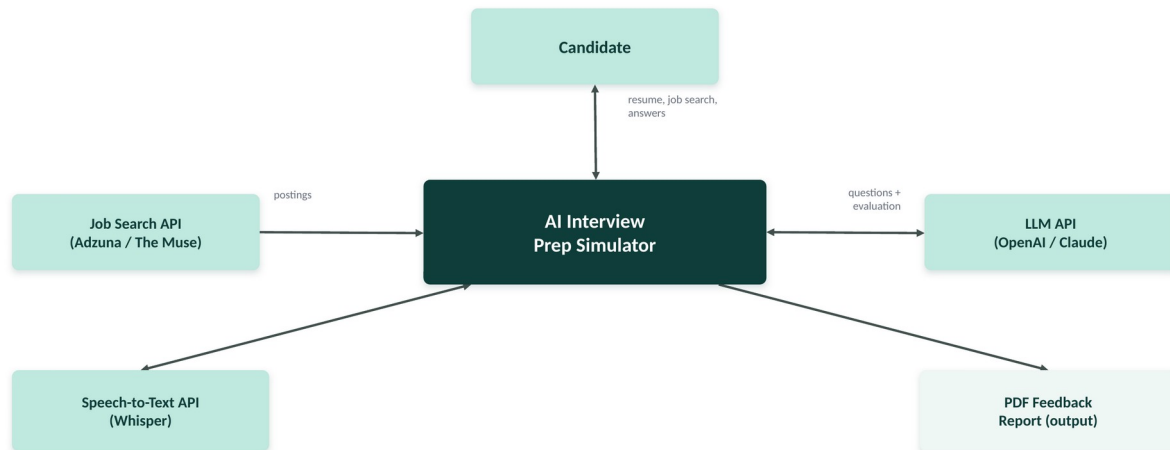


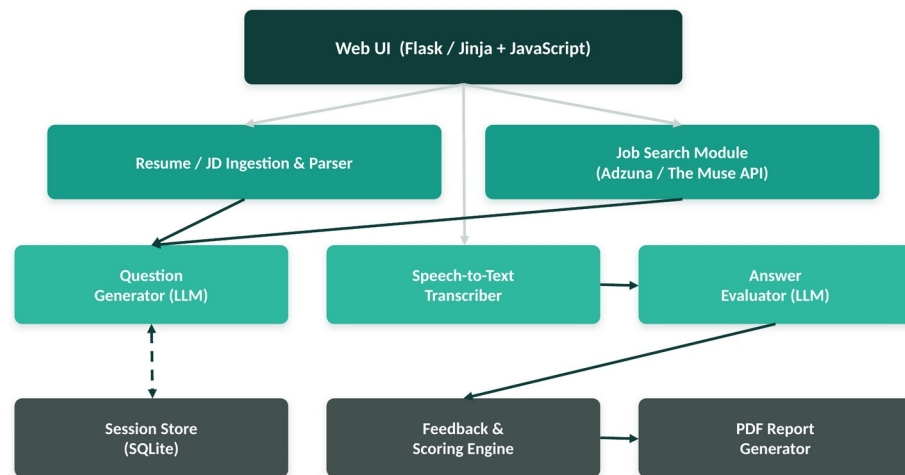
Figure 4: System Context Diagram (C4 model, Level 1: System Context)

7.2 Component Diagram / Building Block View

Figure 5 shows the internal structure of the application as a layered set of components, consistent with the planned source-code structure (one module per component). The Web UI is the single entry point for the candidate and calls both the Resume/JD Ingestion & Parser and the Job Search Module directly; each of those two modules independently feeds its output into the Question Generator (LLM), which combines the resume and job-posting content to produce tailored questions. The Web UI also routes voice answers to the Speech-to-Text Transcriber, whose output feeds the Answer Evaluator (LLM). The Question Generator reads from and writes to the Session Store (dashed arrow), which persists session state across the flow. The Answer Evaluator's scored output feeds the Feedback & Scoring Engine, which in turn supplies the PDF Report Generator with the aggregated results.

The following implementation questions are addressed by this design:

- Which component communicates with the database (Session Store)? The Question Generator reads from and writes to the Session Store.
- Which components communicate with external APIs? The Job Search Module, Question Generator, Speech-to-Text Transcriber, and Answer Evaluator each communicate with one external API (job search, LLM, or speech-to-text) as shown in Figure 4.
- How does the frontend communicate with the backend? The Web UI (Flask/Jinja + JavaScript) submits HTTP requests to Flask backend routes, which orchestrate the ingestion, question-generation, transcription, evaluation, and reporting modules and return JSON/HTML responses rendered by the UI.



dashed = session read/write

Figure 5: Component Diagram / Building Block View (C4 model, Level 2/3: Component — informal, adapted for a single-service architecture)

8. Technologies and Tools

Table 8 lists the technologies and tools planned for this project, their purpose, and why each was selected for this specific system rather than in general terms.

Category	Selected Technology / Tool	Purpose in the Project	Justification
Programming Language	Python	Core backend logic, API integration, file parsing	Mature web (Flask) and NLP/file-parsing libraries; enables fast solo development
Frontend Framework / UI Framework	HTML, CSS, JavaScript + Flask/Jinja templates	Upload form, search UI, interview session flow, results display	Lightweight, no build tooling required for a small solo project; MediaRecorder API supports voice capture
Backend Framework	Flask	Routing, session orchestration, API integration	Minimal, well-documented Python micro-framework suited to a small, API-driven app built by one developer in a short timeframe
Database	SQLite	Session-scoped storage of questions, answers, and scores	File-based, zero-configuration; sufficient for session-scoped data with no need for a full DBMS
Development Environment / IDE	Visual Studio Code	Primary development environment	Free, widely supported, integrated terminal and Python/JavaScript tooling
Version Control System	Git/GitHub	Source control and change history	Standard for individual and academic project tracking; provides backup and a clear commit history
Testing Tools	pytest (unit tests); manual test cases (Table 9)	Verifying module-level logic and end-to-end flows	pytest is lightweight and integrates directly with a Python/Flask codebase
Deployment Platform	Render (cloud demo) / local Docker container (fallback)	Hosting the working demo for defense	Free tier suitable for a small demo deployment; Docker provides an offline fallback if needed
Other Important Libraries	pdfplumber/PyPDF2, python-docx, OpenAI/Claude API, Whisper API, Adzuna/The Muse API, ReportLab	Resume parsing (FR-01), LLM question generation & evaluation (FR-07, FR-11), speech-to-text (FR-10), job search (FR-02–FR-04), PDF export (FR-15)	Each library directly fulfills a specific functional requirement rather than being a general convenience dependency

Table 8: Technologies and Tools

Note on future deployment: a docker-compose configuration is not required for Phase 1, but the official course portfolio requires all web applications to be delivered with one. This has been factored into the plan for a later phase: WP-09 (Deployment & Final Documentation, Section 5, Table 7) will include writing a Dockerfile for the Flask application and a docker-compose.yml that also provisions the SQLite volume, so the system can be started with a single command for grading.

9. Testing Approach

Testing combines manual testing of end-to-end user flows with unit testing of individual backend modules. Manual test cases (Table 9) verify complete flows against the Must-Have functional requirements defined in Section 4. Unit tests (using pytest) will be written for isolated logic such as file-text extraction, keyword extraction, and score aggregation, and will be included directly in the source code repository during implementation, rather than in this document.

Table 9 provides ten manual test cases, using a consistent set of canonical test data across the related cases: a single sample resume (Resume_JaneDoe_JuniorBackendDev.pdf, a 2-page PDF for a junior backend developer listing Python, Flask, SQL, Git, and two internships), a single sample job posting ("Junior Backend Developer — Python, Flask, REST APIs, SQL, Git, 0–2 years' experience"), and two sample answers — one strong, STAR-formatted answer and one deliberately weak, low-effort answer — reused across the relevant test cases. Every Must-Have functional requirement critical to the core flow has at least one associated test case.

TC ID	Related Req. ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result
TC-01	FR-01	Upload a valid PDF resume	User is on the upload screen	1) Select a valid PDF file. 2) Click Upload.	Resume_JaneDoe_JuniorBackendDev.pdf (2-page PDF, junior backend developer: Python, Flask, SQL, Git, two internships)	Resume text is extracted and shown/confirmed within 5 seconds
TC-02	FR-02	Search for job postings by keyword	Candidate has completed resume upload	1) Enter "junior backend developer". 2) Click Search.	Keyword: "junior backend developer"	At least 1 matching posting is displayed within 5 seconds
TC-03	FR-05	Manually enter a job description	Candidate has uploaded a resume and chosen "Enter manually"	1) Paste the job description text. 2) Click Continue.	Junior Backend Developer — Python, Flask, REST APIs, SQL, Git, 0–2 years' experience	Job description text is accepted and shown/confirmed
TC-04	FR-06	Extract role and skill information	Candidate has uploaded the resume and entered/selected a job description	1) Click "Extract role & skills".	Resume_JaneDoe_JuniorBackendDev.pdf (2-page PDF, junior backend developer: Python,	At least 5 key skills/terms are identified (e.g., Python, Flask, REST APIs,

TC ID	Related Req. ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result
					Flask, SQL, Git, two internships) + Junior Backend Developer — Python, Flask, REST APIs, SQL, Git, 0–2 years' experience	SQL, Git) and a role is detected
TC-05	FR-07	Generate interview questions from resume + job posting	Candidate has selected/entered a job posting	1) Click "Start Interview".	Uploaded resume + selected posting	At least 5 questions are generated, mixing behavioral and technical types
TC-06	FR-09	Submit a typed answer	Candidate is on an active question	1) Type an answer. 2) Click Submit.	one strong, STAR-formatted sample answer	Answer is accepted and submitted for evaluation
TC-07	FR-11	Evaluate a submitted text answer	Candidate has submitted an answer	1) Wait for evaluation to complete.	one strong, STAR-formatted sample answer and one deliberately weak, low-effort sample answer (reused across evaluation test cases)	A numeric score and written feedback are returned within 15 seconds; the weak answer receives a visibly lower score than the strong answer
TC-08	FR-12	Display per-question feedback	An answer has been evaluated	1) View the feedback panel for the answered question.	Feedback generated for one strong, STAR-formatted sample answer	Score, strengths/weaknesses, and at least one improvement suggestion

TC ID	Related Req. ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result
						n are displayed
TC-09	FR-13	Calculate overall readiness score	All questions in the session have been answered and scored	1) View the session summary.	A completed session with a mix of strong and weak answers	An overall 0-100 readiness score is displayed, consistent with the mean of the per-question scores
TC-10	FR-16	Handle an unsupported file upload	Candidate is on the upload screen	1) Select a .txt file. 2) Click Upload.	resume_notes.txt (sample file included in the GitHub repository)	A clear error message is shown; the application does not crash

Table 9: Test Cases

10. Glossary

The glossary below is sorted alphabetically and includes terms from requirements, design, testing, the technologies used, and the interview/recruiting application domain.

Term	Definition
Acceptance Criterion	A specific, testable condition that must be satisfied for a requirement to be considered complete.
API (Application Programming Interface)	A defined interface that allows one software system to request data or functionality from another.
Behavioral Question	An interview question that asks a candidate to describe how they handled a past situation, often evaluated using the STAR method.
Candidate	The primary user of the system; a job seeker preparing for an interview.
Context Diagram	A diagram showing a system as a single unit and its interactions with external users and systems.
Domain Model	A diagram representing the key entities, attributes, and relationships within a system's problem domain.
FR (Functional Requirement)	A requirement describing a specific behavior or function the system must provide.
Job Posting	A published description of an open role, including title, company, and required skills, sourced from a job search API or entered manually.
LLM (Large Language Model)	An AI model trained on large volumes of text, used in this project to generate interview questions and evaluate answers.
MoSCoW	A requirements prioritization technique classifying requirements as Must Have, Should Have, Could Have, or Won't Have This Time.
NFR (Non-Functional Requirement)	A requirement describing a quality attribute of the system, such as performance, security, or usability.
Readiness Score	An aggregate numeric score (0–100) summarizing a candidate's performance across all answered questions in a session.
Resume	A candidate-provided document (PDF or DOCX) summarizing work experience and skills, used as input to question generation.
Session	A single, self-contained interview-practice interaction between a candidate, one resume, and one job posting.
Speech-to-Text	The automated process of converting recorded audio into written text, used to transcribe voice answers.
STAR Method	A structured technique (Situation, Task, Action, Result) for answering behavioral interview questions.
Technical Question	An interview question that assesses a candidate's role-specific knowledge or skills.
Use Case Diagram	A UML diagram showing the interactions between actors and the functions (use cases) a system provides.
Waterfall Model	A sequential software development methodology in which each phase must be completed before the next begins.

Table 10: Glossary

11. References

References are formatted in APA style, as required by IU guidelines.

Adzuna. (n.d.). Adzuna API documentation. Retrieved August 13, 2026, from <https://developer.adzuna.com/>

Anthropic. (n.d.). Claude API documentation. Retrieved August 13, 2026, from <https://docs.claude.com/>

Careerflow. (n.d.). Mock interview: Top 10 benefits you must know. Retrieved August 13, 2026, from <https://www.careerflow.ai/blog/mock-interview-benefits>

Flask. (n.d.). Flask documentation. Retrieved August 13, 2026, from <https://flask.palletsprojects.com/>

LinkedIn. (2026). Research: Talent 2026. LinkedIn News. Retrieved August 13, 2026, from <https://news.linkedin.com/2026/LinkedIn-Research-Talent-2026>

Object Management Group. (2017). Unified Modeling Language (UML), version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/>

OpenAI. (n.d.). OpenAI API documentation. Retrieved August 13, 2026, from <https://platform.openai.com/docs>

OpenAI. (n.d.). Whisper speech-to-text guide. Retrieved August 13, 2026, from <https://platform.openai.com/docs/guides/speech-to-text>

RecruitBPM. (2026). 50+ job interview statistics every recruiter must know in 2026. Retrieved August 13, 2026, from <https://recruitbpm.com/blog/40-job-interview-statistics-you-need-to-know>

ReportLab. (n.d.). ReportLab user guide. Retrieved August 13, 2026, from <https://www.reportlab.com/docs/reportlab-userguide.pdf>

The Muse. (n.d.). The Muse API documentation (v2). Retrieved August 13, 2026, from <https://www.themuse.com/developers/api/v2>